PUCRS ESCOLA POLITÉCNICA

98G08-04

CONFIABILIDADE E SEGURANÇA DE SOFTWARE

Conceitos de Dependabilidade

Prof. Iaçanã Ianiski Weber

MARISTA

Baseado no material gentilmente cedido pelos Profs. ***Dr. Alexandre Amory & Dr. Rafael Garibotti***

CONCEITOS

Dependabilidade (*Dependability*):

- ✓ Capacidade de evitar falhas de serviço que são mais frequentes ou mais graves do que o aceitável
  - *"Ability to avoid service failures that are more frequent or more severe than is acceptable"*

Aceita-se o fato que **QUALQUER** sistema falha, mas ele ainda pode ser chamado de ***DEPENDABLE***

CONCEITOS

Falha (*Fault*):

- ✓ É uma **definição incorreta** (dado/instrução incorreta) que pode causar um erro
- ✓ Pode ter origem interna ou externa
- ✓ É um conceito estático, ou seja, está associado ao programa e não a sua execução

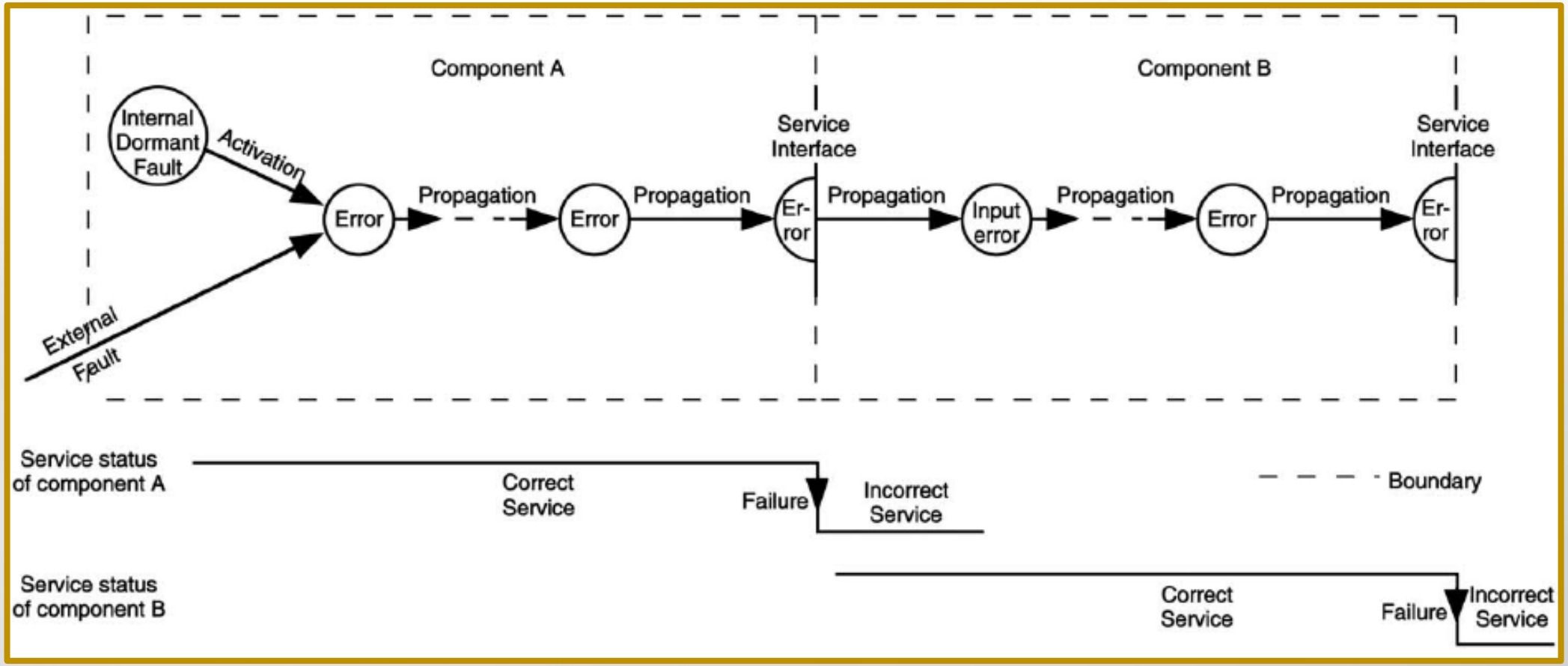
Erro (*Error*):

- ✓ É o **estado interno** diferente do especificado, causado por uma falha, que pode (ou não) virar um defeito

Defeito (*Failure*):

- ✓ É o **estado externo**, ou seja, **efeito que o usuário percebe**, que diverge do resultado esperado: tela azul, *software crash*, *car crash*, ...

Falha (*Fault*) → Erro (*Error*) → Defeito (*Failure*)



Fonte: Avizienis et al. ***Basic Concepts and Taxonomy of Dependable and Secure Computing***. 2004.

ANALOGIA COM MEDICINA

1. Paciente entra no consultório com certos sintomas (Defeitos – *Failures*)
2. O médico deve encontrar a causa dos sintomas (Falhas – *Faults*)
3. Para ajudar no diagnóstico, o médico pede exames para verificar condições internas anômalas (Erros – *Errors*)

✓ A maioria dos problema médicos surgem de ataques externos (vírus, bactérias) ou por degradação física causada pelo envelhecimento

Fonte: Paul Ammann and Jeff Offutt. ***Introduction to Software Testing***. 2008. (pg. 29)

CONCEITOS PARA SOFTWARE

Falha (*Fault*):

- ✓ *Software Fault:* Uma **falha estática** no software

Erro (*Error*):

- ✓ *Software Error:* Um **estado interno incorreto** que é a manifestação de algum defeito

Defeito (*Failure*):

- ✓ *Software Failure:* **Comportamento externo incorreto** em relação aos requisitos ou outra descrição do comportamento esperado

*Fonte: Paul Ammann and Jeff Offutt. ***Introduction to Software Testing***. 2008. (pg. 29)

Falha (*Fault*):

- ✓ É uma **definição incorreta** (dado/instrução incorreta) que pode causar um erro
- ✓ Pode ter origem interna ou externa

Exemplo de Software:

- ✓ Uma variável não inicializada
- ✓ Um acesso a um vetor onde não se verifica o limite do vetor
- ✓ Uma variável com valor fora dos limites especificados

Exemplo de Hardware:

- ✓ Um curto-circuito entre dois sinais
- ✓ Um *bit flip* de uma FSM causado por uma interferência externa

Falha (*Fault*):

- ✓ Falhas podem estar ativas ou dormentes
  - Exemplo dormente: *bug* em uma parte do software que é raramente ativada
- ✓ Podem ter várias origens:
  - De produção: de hardware ou de compilador
  - *Bug*: de hardware e software
  - Omissão: ausência de *feature* especificada para o projeto
  - Malicioso: ataques explorando vulnerabilidade de hardware ou software
- ✓ Em Hardware:
  - Problemas de fabricação, envelhecimento, desgaste, distúrbio externo, ...

ERRO

Erro (***Error***):

- ✓ É o **estado interno** diferente do especificado, causado por uma falha, que pode (ou não) virar um defeito

Exemplo de Software:

- ✓ Acesso a uma posição de memória inválida
  - Pode ter sido causada pela falha de "não verificar o limite do vetor"

Exemplo de Hardware:

- ✓ Um processador que leu uma instrução inválida de memória
  - Pode ter sido causada pela falha de um "*bit flip* na FSM"

Erro (*Error*):

- ✓ Erro pode ser detectado ou latente
- ✓ Exemplo em um processador:
  - Um somador calcula um valor errado
  - O erro vai ficar latente até que o valor calculado erroneamente seja usado

DEFEITO

Defeito (***Failure***):

- ✓ É o **efeito que o usuário percebe**: tela azul, *software crash*, *car crash*, ...

Exemplo de Software:

- ✓ O **erro** de acessar uma posição inválida de memória pode desencadear um **defeito** de travamento do software

Exemplo de Hardware:

- ✓ Vamos assumir um sistema embarcado em um automóvel
- ✓ O **erro** de ler uma instrução inválida pode desencadear um **defeito** no sistema de frenagem, que pode desencadear um acidente, que pode desencadear ...

Defeito (*Failure*):

- ✓ Tipos de defeitos:
  - De serviço: entrega um serviço incorreto
- ✓ Pode ser:
  - Defeito de Conteúdo
  - Defeito de *Timing*: entregou em um tempo não especificado
    - Sistema de tempo real
- ✓ As consequências do defeito:
  - Redução de disponibilidade do serviço
  - Afetar o tempo de resposta do serviço
  - Redução de integridade ou qualidade do serviço
  - Afetar vidas, patrimônio, e ambiente

EXEMPLOS COM SOFTWARE

EXEMPLO DE SOFTWARE

Especificação:

- ✓ Função que conta o número de zeros de um vetor
- ✓ Onde está a **falha** ?!?! Como ela pode causar um **erro** ?

```
int countZero(vector<int> array){
      int count=0
      for (int i= 1; i < array.length(); i++){
            if (array[i] == 0)
                  count++;
      }
      return count;
}
```

Fonte: Paul Ammann and Jeff Offutt. ***Introduction to Software Testing***. 2008. (pg. 29)

EXEMPLO DE SOFTWARE

- ✓ Falha: **int i = 1**
- ✓ Erro:
  - *array*[0] nunca é considerado
  - Erro **PODE** ser causado dependendo dos valores de *array*

```
int countZero(vector<int> array){
      int count=0
      for (int i= 1; i < array.length(); i++){
            if (array[i] == 0)
                  count++;
      }
      return count;
}
```

Fonte: Paul Ammann and Jeff Offutt. ***Introduction to Software Testing***. 2008. (pg. 29)

EXEMPLO DE SOFTWARE

Teste 1:

- ✓ **array = {1, 2, 0}, valor esperado '1'**
  - **Não causa erro** na função, retornando o valor esperado

```
int countZero(vector<int> array){
      int count=0
      for (int i= 1; i < array.length(); i++){
            if (array[i] == 0)
                  count++;
      }
      return count;
}
```

Fonte: Paul Ammann and Jeff Offutt. ***Introduction to Software Testing***. 2008. (pg. 29)

EXEMPLO DE SOFTWARE

Teste 2:

- ✓ **array = {0, 1, 2}, valor esperado '1'**
  - **Erro**: valor retornado == 0

```
int countZero(vector<int> array){
      int count=0
      for (int i= 1; i < array.length(); i++){
            if (array[i] == 0)
                  count++;
      }
      return count;
}
```

Fonte: Paul Ammann and Jeff Offutt. ***Introduction to Software Testing***. 2008. (pg. 29)

EXERCICIOS

```c
// count positive elements in arr
int countPositive(int *arr, int size){
    int count = 0;
    for(int i = 0; i < size; i++){
        if(arr[i] >= 0){
            count++;
        }
    }
    return count;
}
```

```c
// returns last index of valor in arr; -1 if absent
int findLast(int *arr, int size, int valor){
    for(int i = size; i > 0; i--){
        if(arr[i] == valor){
            return i;
        }
    }
    return -1;
}
```

```c
// count odd OR positive values in arr
int oddOrPos(int *arr, int size){
    int count = 0;
    for(int i = 0; i < size; i++){
        if(arr[i] % 2 == 1 || x[i] > 0){
            count++;
        }
    }
    return count;
}
```

```c
// returns last index of 0 in arr; - 1 if absent
int lastZero(int *arr, int size){
    for(int i = 0; i < size; i++){
        if(arr[i] == 0){
            return i;
        }
    }
    return -1;
}
```

DEPENDABILIDADE

ATRIBUTOS DE DEPENDABILIDADE

Dependabilidade (*Dependability*):

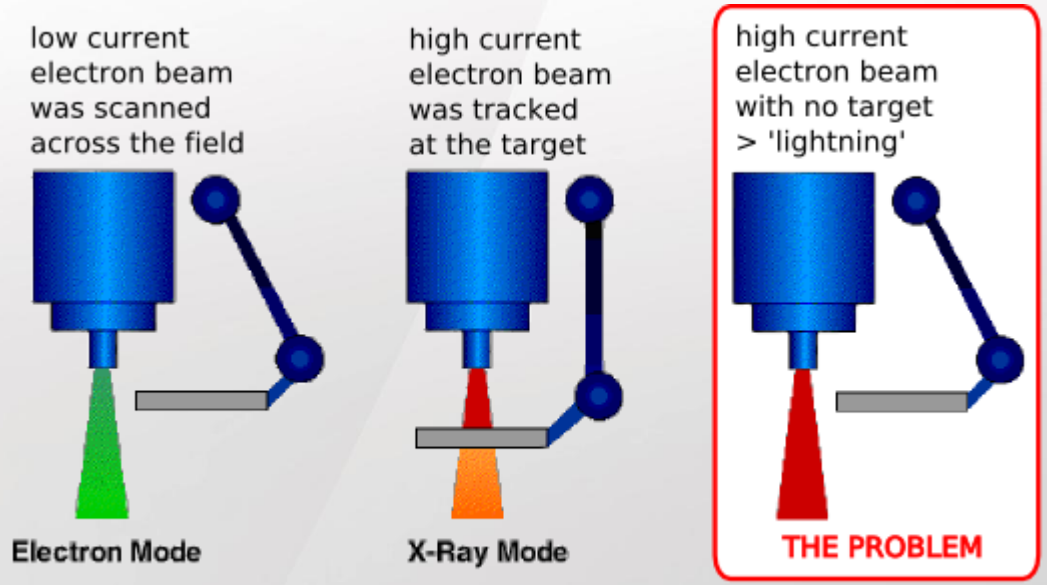
- ✓ ***Availability***: *readiness for correct service*
  - Prontidão de serviço correto
  - O sistema funciona quando eu precisar dele
- ✓ ***Reliability***: *continuity of correct service*
  - Continuidade de serviço correto
  - O sistema continua funcionando
- ✓ ***Safety***: *absence of catastrophic consequences on the user(s) and the environment*
  - Ausência de consequências catastróficas
  - O sistema não vai me machucar ou causar danos
- ✓ ***Security***: *ability of the system to protect itself against deliberate or accidental intrusion*
  - Capacidade de se proteger contra invasões
- ✓ ***Resilience***: *ability to resist and recover from damaging events*
  - Capacidade de se recuperar de eventos causadores de dano

FORMAS DE ALCANÇAR DEPENDABILIDADE

- ✓ Técnicas de prevenção de falhas
  - Objetivo: Metodologias rigorosas empregadas que evitem a criação de bugs!
  - Software: desenvolvimento rigoroso, métodos formais, **testes**, métodos de controle de qualidade → escrever código livre de *bugs*
- ✓ Técnica de tolerância a falhas
  - Lidar com falhas/erros em tempo de execução
  - Apresentar o serviço desejado mesmo na presença de falhas e erros
  - Métodos baseados em **redundância**, em HW e SW
- ✓ Técnicas de remoção de falhas
  - Remoção de falhas durante o desenvolvimento → **testes**
- ✓ Técnicas de predição de falhas
  - Estimar a incidência futura e a consequência das falhas e erros
  - Tentar antecipar falhas e erros

EXEMPLOS DE FALHAS DE DEPENDABILIDADE

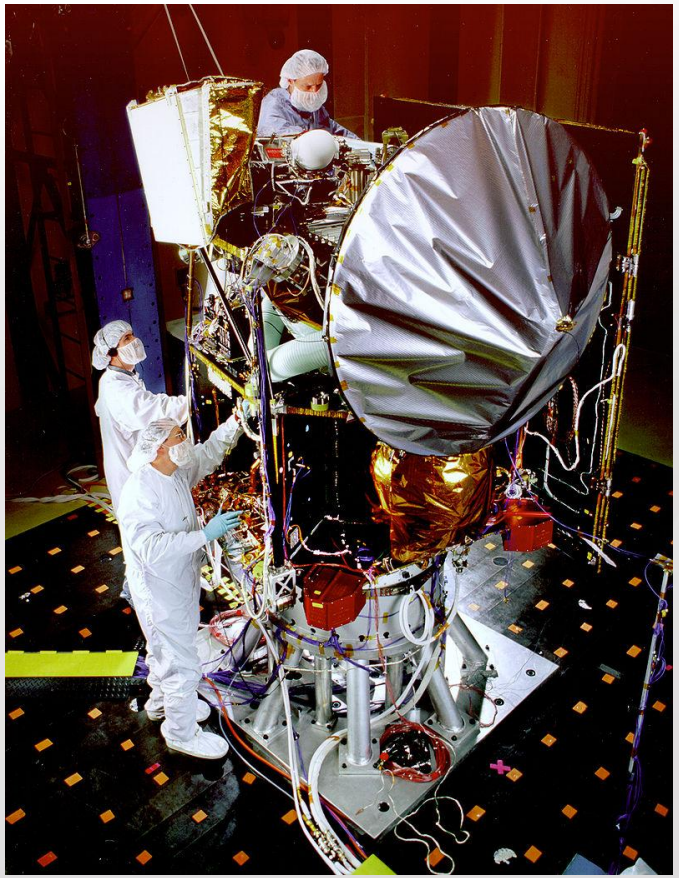
- ✓ Máquina de radioterapia **Therac-25** – 1985-1987
  - o software da máquina não detectava que o *target* não estava corretamente posicionado
  - https://pt.wikipedia.org/wiki/Therac-25



tray including the target, a flattening filter, the collimator jaws and an ion chamber was moved OUT for "electron" mode, and IN for "photon" mode.

"A AECL nunca havia testado o Therac-25 com a combinação de software e hardware até que ele fosse montado no hospital."

- ✓ **Sonda Mars** - Setembro de 1999
  - Um módulo computou dados do propulsor em unidades inglesas e encaminhou os dados para um módulo que esperava dados em unidades métricas
  - https://www1.folha.uol.com.br/fsp/ciencia/fe0110199905.htm



Custo:
U$327.6 milhões
(U$515.39 milhões em 2021)

FALHAS DE SISTEMAS

- **Patriot Missile Launcher**
  - Lançador de mísseis móvel, **projetado para algumas horas de operação**
  - Usado para operação de defesa *Scud*, **nunca projetado para isso**
  - Fevereiro de 1991 - Bateria em Dharan (Arábia Saudita) não conseguiu interceptar o míssil *Scud*, atingiu o exército (28 vítimas fatais)
  - Problema de envelhecimento de software no computador de controle de armas do sistema
  - Velocidade e tempo de destino registrados como inteiros de 24 bits
  - Cálculo de rastreamento impreciso (erro de 0.34s) devido a operação muito longa (> 100 horas)
  - *Software modificado chegou à base um dia após o acidente*

https://www-users.cse.umn.edu/~arnold/disasters/patriot.html

FALHAS DE SOFTWARE

- ✓ **Foguete Ariane 5** - Explodiu 37 segundos após a decolagem em 1996
  - Exceção não tratada de conversão de ponto flutuante
  - http://www-users.math.umn.edu/~arnold/disasters/ariane.html

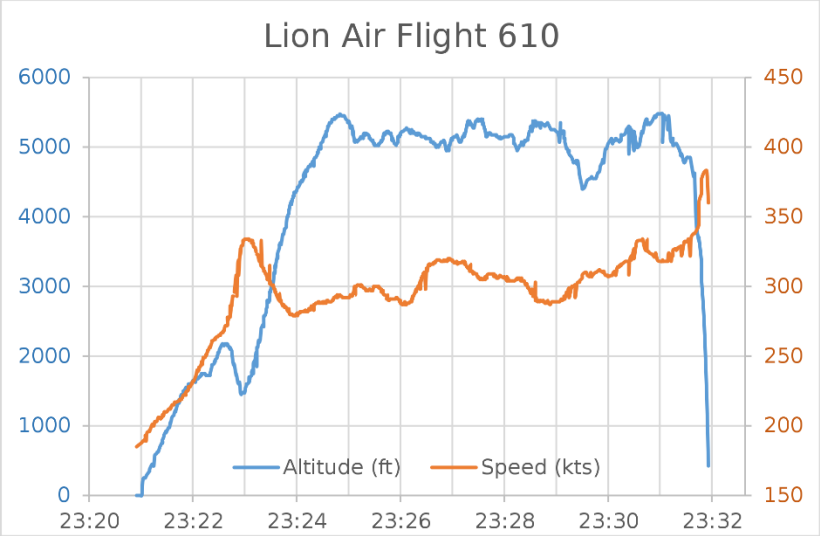
https://www.youtube.com/watch?v=qnHn8W1Em6E&ab_channel=MrSifta

- ✓ Um overflow de mais de 1 bilhão de dólares!

Somente casos antigos???

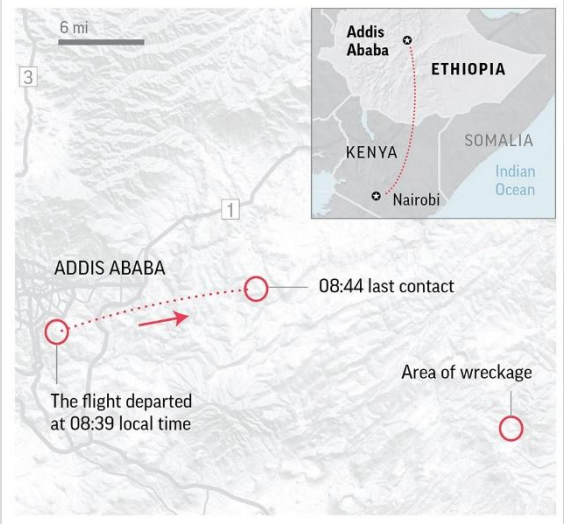
Lion Air Flight 610 – Outubro 2018

- 13 minutos de voo
- 189 vítimas

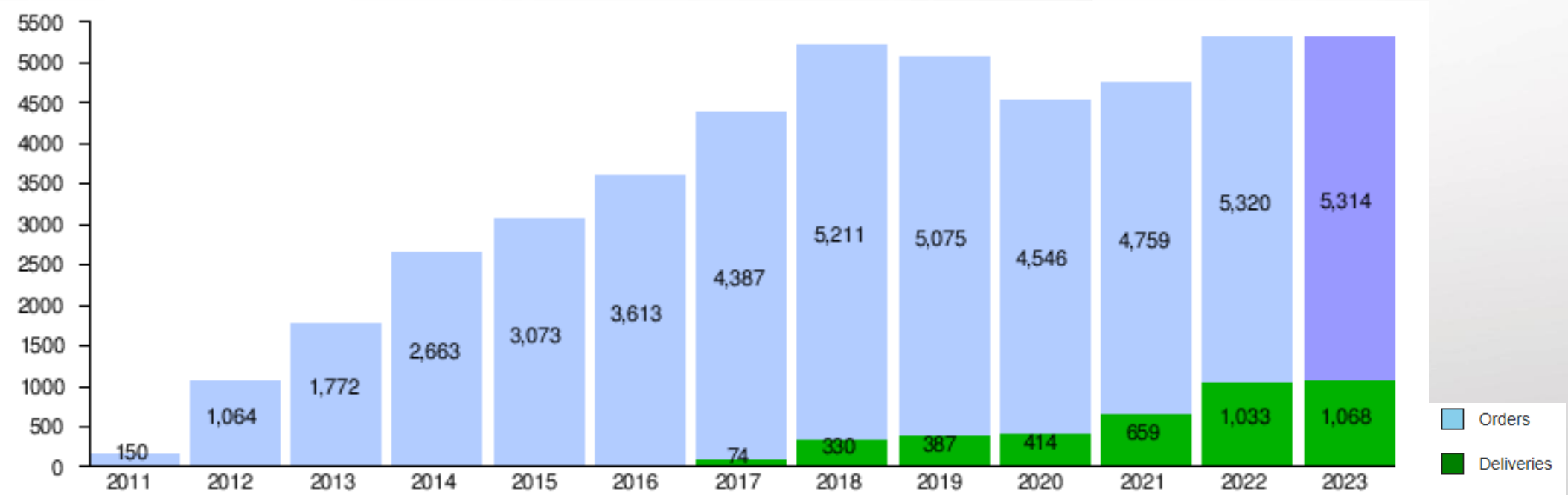


- 6 minutos de voo
- 157 vítimas

Ethiopian Airlines Flight 302 – Março 2019



Boeing 737 MAX



costs of the MAX groundings were US$20 billion and indirect costs over US$60 billion

https://edition.cnn.com/2020/11/17/business/boeing-737-max-grounding-cost/index.html

Boeing 737 MAX - histórico

- Projeto base concebido em 1959

737 - 200

- Trens de pouso baixos!

Boeing 737 MAX - histórico

- Projeto base concebido em 1959

737 - 300

- Trens de pouso baixos!

Boeing 737 MAX - histórico

737 – 600/700/800/900 (NG)

Aircraft	Orders			Deliveries																											
Model series	Total	Unfilled	Total	2023	2022	2021	2020	2019	2018	2017	2016	2015	2014	2013	2012	2011	2010	2009	2008	2007	2006	2005	2004	2003	2002	2001	2000	1999	1998	1997
737-600	69	—	69	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—	10	3	3	6	5	4	6	24	8	—	—
737-700	1,128	—	1,128	—	—	—	—	—	2	4	6	7	11	12	7	43	23	51	61	101	103	93	109	80	71	85	75	96	85	3
737-700C	5	—	5	—	—	—	—	—	—	—	2	—	—	—	—	—	—	—	1	—	—	—	—	—	—	2	—	—	—	—
737-800	4,991	2	4,989	—	—	—	2	28	269	397	411	396	386	347	351	292	323	283	190	214	172	104	78	69	126	168	185	133	65	—
737-900	52	—	52	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—	6	6	11	8	21	—	—	—	—
737-900ER	505	—	505	—	—	—	—	22	34	37	52	73	70	67	44	24	15	28	30	9	—	—	—	—	—	—	—	—	—	—
(Commercial Jets)	6,750	2	6,748	—	—	—	2	50	305	438	471	476	467	426	402	359	361	362	282	334	278	206	199	165	209	282	284	237	150	3
737-700W	14	—	14	—	—	—	—	—	—	—	—	—	—	—	—	—	2	2	—	—	5	2	1	1	1	—	—	—	—	—
737-800A	191	23	168	—	12	16	14	17	18	17	18	15	13	8	9	5	1	3	2	—	—	—	—	—	—	—	—	—	—	—
C-40A	17	—	17	—	—	—	2	—	—	1	—	—	2	—	—	1	2	—	—	—	1	—	2	—	2	1	3	—	—	—
C-40B/C	9	—	9	—	—	—	—	—	—	—	—	—	—	—	—	1	—	—	—	1	2	—	—	2	1	1	1	—	—	—
(Military jets)	231	23	208	—	12	16	14	19	18	17	19	15	15	8	9	7	5	5	2	1	8	2	3	3	4	2	4	—	—	—
BBJ 737-700	113	—	113	—	1	—	—	1	1	—	—	3	1	5	2	6	4	4	4	5	7	3	3	1	7	12	10	25	8	—
BBJ 737-800	23	—	23	—	—	2	—	—	—	—	—	—	2	1	2	—	2	—	1	—	2	1	—	3	2	5	—	—	—	—
BBJ 737-900	7	—	7	—	—	—	—	—	—	—	—	1	—	—	—	—	4	1	1	—	—	—	—	—	—	—	—	—	—	—
(Business jets)	143	—	143	—	1	2	—	1	1	—	—	4	3	6	4	6	10	5	6	5	9	4	3	4	9	17	10	25	8	—
(737 NextGen)	7,124	25	7,099	—	13	18	16	70	324	455	490	495	485	440	415	372	376	372	290	330	302	212	202	173	223	299	280	278	166	3

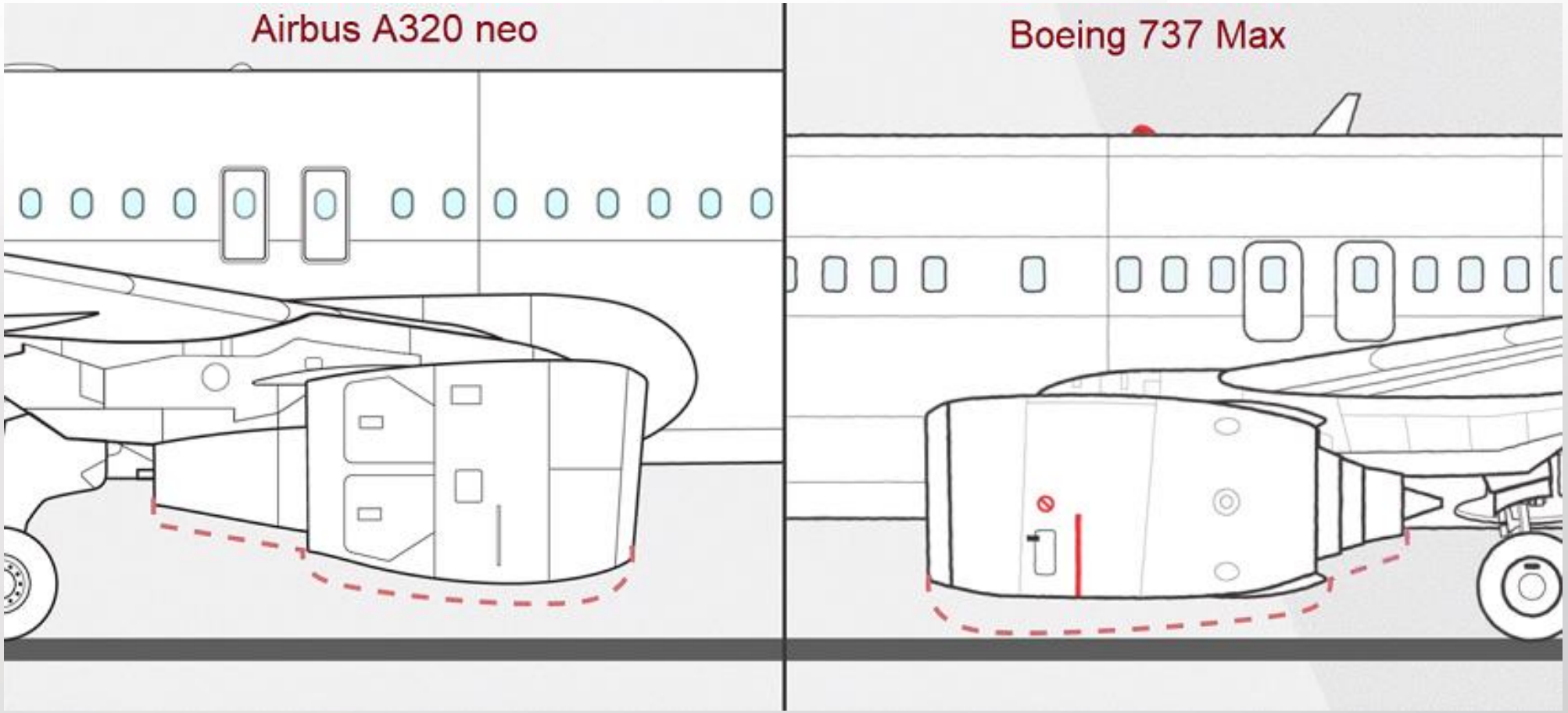
~21300 aviões entregues

Dados até Jan. 2023

Boeing 737 MAX - histórico

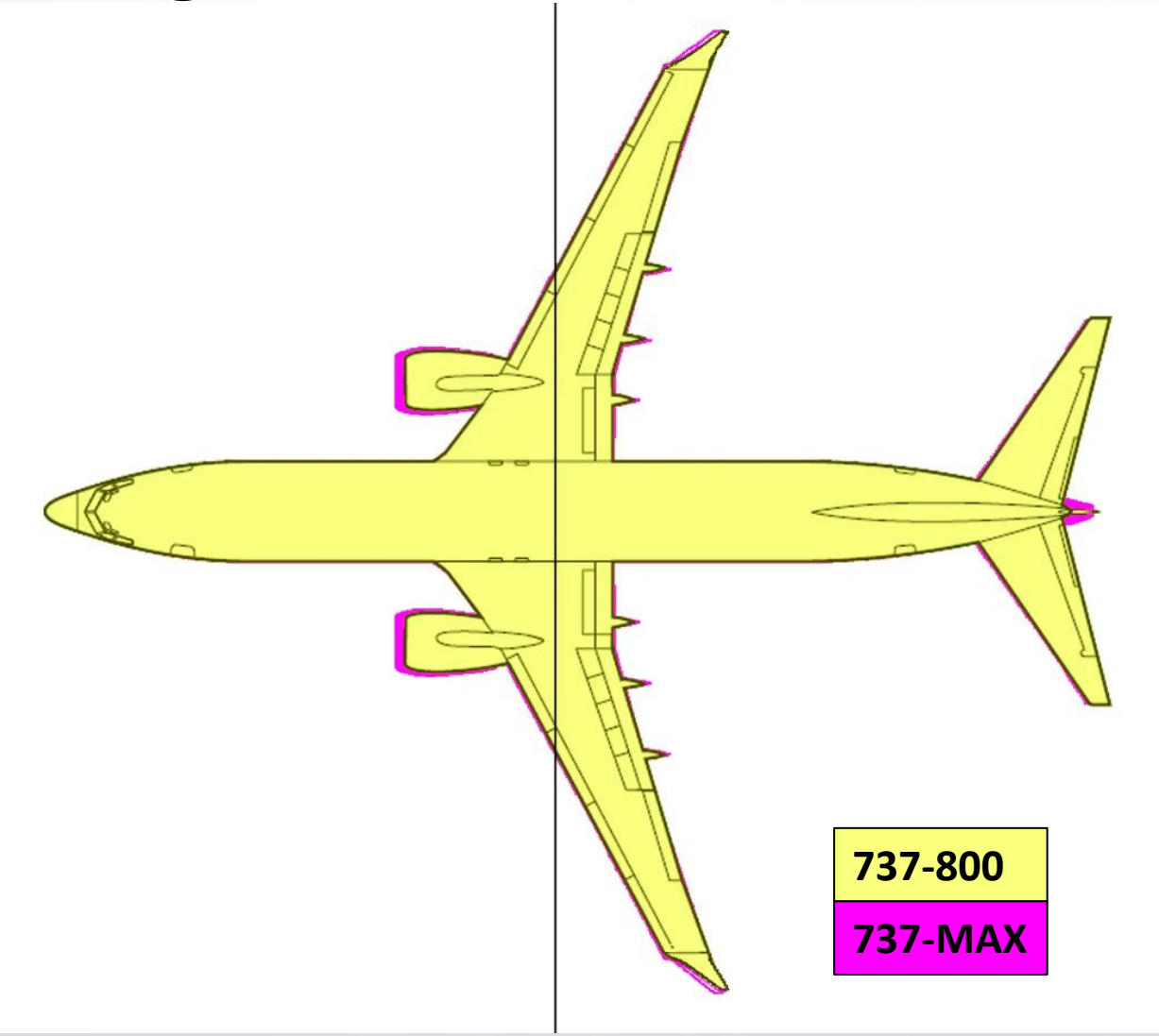
Airbus A320neo (New Engine Option)

Boeing 737 MAX - histórico



Boeing 737 MAX - histórico

Boeing 737 MAX - histórico



Até então o controle de voo era naturalmente estável, sem necessidade de correções na inclinação (*pitch*)

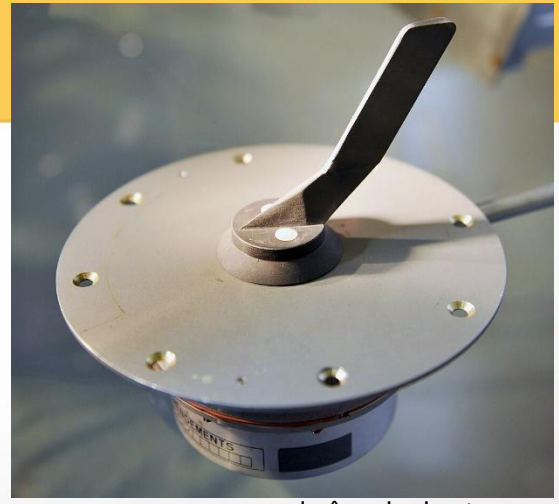
Em condições normais de inclinação o efeito de desestabilização não é relevante

Para manobras onde o ângulo de ataque é elevado, o desequilíbrio causado pelos novos motores poderia levar o avião ao *stall* (perda de sustentação)

Boeing introduz o MCAS (Sistema de Aumento de Características de Manobra) – **SOFTWARE**

Com base na velocidade indicada, na altitude e sensores de ângulo de ataque, ele abaixa o nariz quando a ângulo de ataque da aeronave é muito íngreme, para prevenir condições de *stall* aerodinâmico, sem a ação do piloto.

Boeing 737 MAX - histórico



sensor de ângulo de ataque

1/13/2023

National Transportation Safety Board
Response to Final Aircraft Accident Investigation Report
Ethiopian Airlines Flight 302
Boeing 737-8 MAX, ET-AVJ
Ejere, Ethiopia
March 10, 2019

In accordance with Annex 13 to the Convention on International Civil Aviation, as the accredited representative of the State of Design and Manufacture of the airplane, the National Transportation Safety Board (NTSB) participated in the Aircraft Accident Investigation Bureau of Ethiopia's (EAIB) investigation of the subject accident. Technical advisors from Boeing, Collins Aerospace, and the Federal Aviation Administration assisted the NTSB, as provided by Annex 13.

On December 23, 2022, the EAIB issued its final accident report on the Ethiopian Airlines flight 302 accident. Overall, the NTSB concurs with the EAIB's investigation of the Maneuvering Characteristics Augmentation System (MCAS) and related systems and the roles that they played in the accident.[1] However, the NTSB's comments (dated May 12, 2022) on the EAIB's last draft report (dated March 30, 2022) were not appended to the final report, as requested by the NTSB and provided by section 6.3 of Annex 13.[2] Instead, the EAIB's final report included a hyperlink to an earlier and outdated version of the NTSB's comments. In response, on December 27, 2022, the NTSB published a news release and the report comments that were most recently provided to the EAIB.

The news release indicated that the EAIB's final accident report included significant changes from the last report draft and that the NTSB was carefully reviewing the final report and determining whether any other comments might be necessary. The NTSB's review found that the final report contained new information that the EAIB had not afforded the NTSB the opportunity to review before the report was issued.

According to the EAIB's final report, electrical problems that existed since the time of the accident airplane's production caused the left angle-of-attack (AOA) sensor heater to fail, which resulted in the AOA sensor providing erroneous values that caused the MCAS to pitch the nose of the airplane downward, resulting in

[1] MCAS is an extension of the Boeing 737 speed trim system. It was designed to improve airplane handling characteristics and decrease pitch-up tendencies at elevated angles of attack. As the development of the Boeing 737 MAX progressed, the MCAS function was expanded to low Mach numbers. MCAS was also designed to function only during manual flight (autopilot not engaged), with the airplane's flaps up, and at an elevated angle of attack.

[2] The NTSB received the first draft accident report on January 12, 2021, and provided its comments to the EAIB on February 26, 2021. The NTSB received the second draft accident report on May 26, 2021, and provided its comments to the EAIB on June 11, 2021.

"problemas elétricos que existiam desde o momento da produção do avião acidentado causaram a falha do aquecedor do sensor de ângulo de ataque (AOA) esquerdo, o que resultou no sensor AOA fornecendo valores errados que fizeram com que o MCAS inclinasse o nariz do avião para baixo"

"Se os defeitos intermitentes não tivessem causado a falha dos sensores AOA no voo do acidente, o MCAS não teria sido ativado e o acidente não teria ocorrido. O MCAS teria permanecido como [uma] ameaça oculta até que sua verdadeira natureza fosse exposta por alguma outra causa válida ou errônea."

https://www.ntsb.gov/investigations/Documents/Response%20to%20EAIB%20final%20report.pdf